



GitHub

GIT CHEAT SHEET

Git is the free and open source distributed version control system that's responsible for everything GitHub related that happens locally on your computer. This cheat sheet features the most important and commonly used Git commands for easy reference.

INSTALLATION & GUI

With platform specific installers for Git, GitHub also provides the ease of staying up-to-date with the latest releases of the command line tool while providing a graphical user interface for day-to-day interaction, review, and repository synchronization.

GitHub for Windows

<https://windows.github.com>

GitHub for Mac

<https://mac.github.com>

For Linux and Solaris platforms, the latest release is available on the official Git web site.

Git for All Platforms

<http://git-scm.com>

SETUP

Configuring user information used across all local repositories

```
git config --global user.name "[firstname lastname]"
```

set a name that is identifiable for credit when review version history

```
git config --global user.email "[valid-email]"
```

set an email address that will be associated with each history marker

```
git config --global color.ui auto
```

set automatic command line coloring for Git for easy reviewing

SETUP & INIT

Configuring user information, initializing and cloning repositories

```
git init
```

initialize an existing directory as a Git repository

```
git clone [url]
```

retrieve an entire repository from a hosted location via URL

STAGE & SNAPSHOT

Working with snapshots and the Git staging area

```
git status
```

show modified files in working directory, staged for your next commit

```
git add [file]
```

add a file as it looks now to your next commit (stage)

```
git reset [file]
```

unstage a file while retaining the changes in working directory

```
git diff
```

diff of what is changed but not staged

```
git diff --staged
```

diff of what is staged but not yet committed

```
git commit -m "[descriptive message]"
```

commit your staged content as a new commit snapshot

BRANCH & MERGE

Isolating work in branches, changing context, and integrating changes

```
git branch
```

list your branches. a * will appear next to the currently active branch

```
git branch [branch-name]
```

create a new branch at the current commit

```
git checkout
```

switch to another branch and check it out into your working directory

```
git merge [branch]
```

merge the specified branch's history into the current one

```
git log
```

show all commits in the current branch's history



Git Cheat Sheet



INSPECT & COMPARE

Examining logs, diffs and object information

git log

show the commit history for the currently active branch

git log branchB..branchA

show the commits on branchA that are not on branchB

git log --follow [file]

show the commits that changed file, even across renames

git diff branchB...branchA

show the diff of what is in branchA that is not in branchB

git show [SHA]

show any object in Git in human-readable format

TRACKING PATH CHANGES

Versioning file removes and path changes

git rm [file]

delete the file from project and stage the removal for commit

git mv [existing-path] [new-path]

change an existing file path and stage the move

git log --stat -M

show all commit logs with indication of any paths that moved

IGNORING PATTERNS

Preventing unintentional staging or committing of files

```
logs/  
*.notes  
pattern*/
```

Save a file with desired patterns as .gitignore with either direct string matches or wildcard globs.

git config --global core.excludesfile [file]

system wide ignore pattern for all local repositories

SHARE & UPDATE

Retrieving updates from another repository and updating local repos

git remote add [alias] [url]

add a git URL as an alias

git fetch [alias]

fetch down all the branches from that Git remote

git merge [alias]/[branch]

merge a remote branch into your current branch to bring it up to date

git push [alias] [branch]

Transmit local branch commits to the remote repository branch

git pull

fetch and merge any commits from the tracking remote branch

REWRITE HISTORY

Rewriting branches, updating commits and clearing history

git rebase [branch]

apply any commits of current branch ahead of specified one

git reset --hard [commit]

clear staging area, rewrite working tree from specified commit

TEMPORARY COMMITS

Temporarily store modified, tracked files in order to change branches

git stash

Save modified and staged changes

git stash list

list stack-order of stashed file changes

git stash pop

write working from top of stash stack

git stash drop

discard the changes from top of stash stack

GitHub Education

Teach and learn better, together. GitHub is free for students and teachers. Discounts available for other educational uses.

✉ education@github.com

🌐 education.github.com



Git and GitHub

Git and GitHub are two related but distinct concepts:

Git:

- Git is a version control system (VCS) that allows developers to track changes made to their code, collaborate with others, and maintain a record of all modifications.
- Git is a decentralized system, meaning that every developer working on a project has a local copy of the entire project history.
- Git provides features like branching, merging, and commit history, making it a powerful tool for managing code changes.

GitHub:-

- GitHub is a web-based platform that provides a centralized location for Git repositories.
- GitHub allows developers to host and manage their Git repositories online, making it easy to collaborate with others and share code.
- GitHub offers features like issue tracking, pull requests, code review, and project management, making it a comprehensive platform for software development.

In summary:

- Git is the version control system that manages code changes.
- GitHub is the platform that hosts and provides a user-friendly interface for Git repositories.

Think of it like a library:-

- Git is like the bookshelves where you store and organize your books (code).
- GitHub is like the library building where you can store, share, and collaborate on your books (code) with others.

For more info visit :-

[Git and GitHub | Chai Docs \(chaicode.com\)](https://chaicode.com/)



git add .

Here's what `git add .` does in Git:

`git add .` is a command that stages all changes in the current directory and subdirectories. The dot (.) represents the current directory.

When you run `git add .`, Git stages:

1. New files
2. Modified files
3. Deleted files

This command is a convenient way to stage all changes in your working directory without having to specify individual files or directories.

Here's a breakdown of what happens:

- `git add`: This command stages changes for the next commit.
- `.`: The dot represents the current directory, including all subdirectories.

By combining `git add` with `.`, you're telling Git to stage all changes in the current directory and subdirectories.

Example:

Suppose you have a project with the following changes:

- New file: `new-file.txt`
- Modified file: `modified-file.txt`
- Deleted file: `deleted-file.txt`

Running `git add .` will stage all these changes, preparing them for the next commit.

Remember to use `git status` to verify which changes have been staged and `git commit` to commit the staged changes.



Git status command

The git status command is used to display the state of the working directory and staging area. It shows which changes have been staged, which changes have not been staged, and which files are untracked.

Basic usage:

```
git status
```

This will display the following information:

1. Changes to be committed (staged changes):
 - Files that have been added to the staging area with git add.
2. Changes not staged for commit:
 - Files that have been modified but not yet added to the staging area.
3. Untracked files:
 - Files that are not being tracked by Git.

Options:

- --short or -s: Display a concise version of the status.
- --branch or -b: Display the branch name and its status.
- --porcelain: Display the status in a machine-readable format.

Examples:

- git status -s: Show a concise status.
- git status -b: Show the branch name and its status.
- git status --porcelain: Show the status in a machine-readable format.

Note: git status is a useful command to run frequently to keep track of the changes in your repository.



commit -m command

The `git commit -m` command is used to commit changes with a meaningful commit message.

Here's a breakdown of the command:

- `git commit`: This command commits the staged changes (added using `git add`) to the local repository.
- `-m`: This option allows you to specify a commit message directly on the command line.
- `"<message>"`: The commit message, enclosed in quotes, that describes the changes made in the commit.

Example:

```
git commit -m "Fixed bug in login feature"
```

This command commits the staged changes with a commit message that describes the changes made.

Best practices for commit messages:

1. Keep it concise (less than 50 characters)
2. Use present tense (e.g., "Fix" instead of "Fixed")
3. Describe the change, not the fix (e.g., "Update login feature" instead of "Fixed bug in login feature")

Using `git commit -m` saves time and makes it easier to write meaningful commit messages. Remember to keep your commit messages clear and concise!



Git log command

The git log command is used to display a log of commits made to a Git repository. It shows a list of commits, including the commit hash, author, date, and commit message.

Basic usage:

```
git log
```

This will display a list of commits in reverse chronological order, with the most recent commit first.

Options:

- --oneline: Show each commit on a single line.
- --graph: Display a graphical representation of the commit history.
- --author: Filter commits by author.
- --since/--until: Filter commits by date range.
- --grep: Filter commits by keyword in commit message.
- --all: Show all commits, including those on other branches.

Examples:

- git log --oneline: Show a concise list of commits.
- git log --graph --all: Show a graphical representation of all commits.
- git log --author=John: Show only commits made by John.
- git log --since=1.week.ago: Show commits made in the last week.
- git log --grep=bugfix: Show commits with "bugfix" in the commit message.

Note: You can combine options to create a customized log view.



git rm command

The `git rm` command is used to remove files from the Git repository and the working directory.

Basic usage:

```
git rm <file>
```

This will remove the specified file from the repository and the working directory.

Options:-

- `--cached`: Remove the file from the repository, but not from the working directory.
- `--force` or `-f`: Force the removal of the file, even if it's not tracked by Git.
- `--ignore-unmatch` or `-r`: Remove files recursively, even if they're not tracked by Git.
- `--quiet` or `-q`: Suppress output messages.

Examples:

- `git rm file.txt`: Remove `file.txt` from the repository and working directory.
- `git rm --cached file.txt`: Remove `file.txt` from the repository, but not from the working directory.
- `git rm -f file.txt`: Force removal of `file.txt`, even if it's not tracked by Git.
- `git rm -r directory/`: Remove the `directory/` and all its contents recursively.

Note:

- `git rm` will stage the removal of the file, so you need to commit the changes with `git commit` to finalize the removal.
- Be careful when using `git rm`, as it permanently deletes files from the repository and working directory.



Git reset command

The git reset command is used to reset the current branch to a specific commit or state.

Basic usage:

```
git reset <commit>
```

This will reset the current branch to the specified commit, moving the branch pointer to that commit.

Options

- --soft: Reset the branch pointer to the specified commit, but keep changes in the staging area and working directory.
- --mixed (default): Reset the branch pointer to the specified commit, and reset the staging area to match the commit, but leave changes in the working directory.
- --hard: Reset the branch pointer to the specified commit, and reset the staging area and working directory to match the commit.
- --merge: Reset the branch pointer to the specified commit, and merge changes from the commit into the working directory.
- --keep: Reset the branch pointer to the specified commit, but keep local changes.

Examples:

- git reset HEAD~1: Reset the branch to the previous commit.
- git reset --soft HEAD~1: Reset the branch to the previous commit, but keep changes in the staging area and working directory.
- git reset --hard HEAD~1: Reset the branch to the previous commit, and reset the staging area and working directory to match the commit.
- git reset --merge HEAD~1: Reset the branch to the previous commit, and merge changes from the commit into the working directory.

Note:

- git reset will move the branch pointer to the specified commit, potentially discarding commits.
- Use git reset with caution, as it can rewrite history and discard changes.



Git revert command

The git revert command is used to undo changes introduced by a specific commit.

Basic usage:

```
git revert <commit>
```

This will create a new commit that undoes the changes introduced by the specified commit.

Options:

- --no-commit: Don't create a new commit, only stage the revert.
- --edit: Edit the commit message before committing the revert.
- --mainline: Revert changes on the mainline (parent) commit.
- --no-edit: Don't edit the commit message, use the default message.

Examples:

- git revert HEAD~1: Undo changes introduced by the previous commit.
- git revert <commit-hash>: Undo changes introduced by the specified commit.
- git revert -n <commit-hash>: Stage the revert, but don't create a new commit.

Note:

- git revert creates a new commit, it doesn't modify the original commit.
- git revert only undoes changes introduced by the specified commit, it doesn't affect other commits.
- Use git revert to undo changes in a safe and non-destructive way, without rewriting history.



Git diff command

The git diff command is used to show changes between commits, branches, or files.

Basic usage:

```
git diff
```

This will show changes between the working directory and the staging area.

Options:

- `git diff <commit>`: Show changes between the working directory and the specified commit.
- `git diff <commit> <commit>`: Show changes between the two specified commits.
- `git diff <branch>`: Show changes between the current branch and the specified branch.
- `git diff --staged`: Show changes between the staging area and the last commit.
- `git diff --cached`: Same as `--staged`.
- `git diff <file>`: Show changes in the specified file.

Flags:

- `-w` or `--ignore-all-space`: Ignore whitespace changes.
- `--ignore-space-at-eol`: Ignore changes in whitespace at the end of lines.
- `-b` or `--ignore-space-change`: Ignore changes in the amount of whitespace.
- `--color` or `--color=always`: Show colored output.
- `--stat`: Show a summary of changes.



Git diff command

Examples:

- `git diff HEAD~1`: Show changes between the working directory and the previous commit.
- `git diff master..feature`: Show changes between the master branch and the feature branch.
- `git diff --staged`: Show changes between the staging area and the last commit.
- `git diff -w file.txt`: Show changes in file.txt, ignoring whitespace changes.

Note:

- `git diff` is a useful command to review changes before committing.
- Use `git diff` to see what changes have been made, and to ensure you're committing the correct changes.



git branch

The git branch command is used to manage branches in a Git repository. Here are some common uses:

1. List branches: `git branch`
 - Lists all local branches.
2. Create a new branch: `git branch <branch_name>`
 - Creates a new branch with the given name.
3. Switch to a branch: `git checkout <branch_name>`
 - Switches to the specified branch.
4. Create and switch to a new branch: `git checkout -b <branch_name>`
 - Creates a new branch and switches to it.
5. Delete a branch: `git branch -d <branch_name>`
 - Deletes the specified branch.
6. Rename a branch: `git branch -m <old_branch_name> <new_branch_name>`
 - Renames the specified branch.
7. List remote branches: `git branch -a`
 - Lists all local and remote branches.
8. Push a branch to remote: `git push <remote_name> <branch_name>`
 - Pushes the specified branch to the remote repository.
9. Pull from a branch: `git pull <remote_name> <branch_name>`
 - Pulls changes from the specified branch in the remote repository.

Note:

- git branch only operates on local branches.
- git checkout is used to switch between branches.
- git push and git pull are used to interact with remote branches.



Git checkout command

The git checkout command is used to switch between branches, restore files, and more. Here are some common uses:

1. Switch between branches:

- git checkout <branch-name>: Switch to the specified branch.
- git checkout -b <new-branch-name>: Create a new branch and switch to it.

2. Restore files:

- git checkout <file-name>: Restore the specified file to its last committed state.
- git checkout -- <file-name>: Discard changes to the specified file.

3. Restore directories:

- git checkout <directory-name>: Restore the specified directory to its last committed state.

4. View previous commits:

- git checkout <commit-hash>: Switch to the specified commit.

5. Create a new branch from a commit:

- git checkout -b <new-branch-name> <commit-hash>: Create a new branch from the specified commit.

Some additional options:

- -b: Create a new branch.
- --: Discard changes to a file or directory.
- -f: Force checkout, overwriting local changes.
- --merge: Merge changes from another branch.

Remember to use git status and git log to understand the current state of your repository before using git checkout.



Git switch

git switch is a command used to switch between branches or restore working tree files. It's similar to git checkout, but with a more intuitive syntax.

Common uses:

1. Switch to a branch:

- `git switch <branch-name>`: Switch to the specified branch.

2. Create and switch to a new branch:

- `git switch -c <new-branch-name>`: Create a new branch and switch to it.

3. Switch to a commit:

- `git switch <commit-hash>`: Switch to the specified commit.

4. Restore working tree files:

- `git switch -- <file-name>`: Restore the specified file to its last committed state.

Options:

- `-c`: Create a new branch.

- `-d`: Delete the current branch.

- `--`: Restore working tree files.

- `--orphan`: Create a new orphan branch.

git switch is a more modern and user-friendly alternative to git checkout. It's designed to make it easier to switch between branches and restore files.

Key differences between git switch and git checkout:

- git switch is more explicit and intuitive.

- git switch only updates the working tree and index, while git checkout can also update the HEAD.

- git switch is more restrictive, preventing accidental branch switching.

Remember, git switch is available in Git version 2.23 and later. In earlier versions, use git checkout instead.

✧ git merge

The git merge command is used to combine changes from two or more branches into a single branch. Here are some common uses:

1. Merge a branch into the current branch: `git merge <branch_name>`
 - Merges the specified branch into the current branch.
2. Merge a branch into the master branch: `git checkout master && git merge <branch_name>`
 - Switches to the master branch and merges the specified branch into it.
3. Merge multiple branches: `git merge <branch1> <branch2>`
 - Merges multiple branches into the current branch.
4. Merge a specific commit: `git merge <commit_hash>`
 - Merges a specific commit into the current branch.
5. Merge with no commit: `git merge --no-commit <branch_name>`
 - Merges the specified branch, but does not create a new commit.
6. Merge with squash: `git merge --squash <branch_name>`
 - Merges the specified branch and squashes all commits into a single commit.

Options:

- `--no-ff`: Forces a merge commit even if the merge could be resolved as a fast-forward.
- `--no-commit`: Does not create a new commit.
- `--squash`: Squashes all commits into a single commit.
- `-m`: Specifies a custom merge commit message.

Note:

- git merge can lead to conflicts if changes are made to the same files in both branches.
- Use `git status` and `git diff` to resolve conflicts before committing the merge.



Git stash

git stash is a command that temporarily saves changes you've made to your working directory, allowing you to switch branches or work on a different task without committing your changes.

Common uses:

1. Stash changes:

- git stash: Save changes to your working directory and revert to the last commit.

2. Stash changes with a message:

- git stash save "<message>": Save changes with a descriptive message.

3. List stashed changes:

- git stash list: Show a list of stashed changes.

4. Apply stashed changes:

- git stash apply: Apply the most recent stashed changes.
- git stash apply stash@{<number>}: Apply a specific stashed change.

5. Drop stashed changes:

- git stash drop: Delete the most recent stashed changes.
- git stash drop stash@{<number>}: Delete a specific stashed change.

6. Pop stashed changes:

- git stash pop: Apply and delete the most recent stashed changes.

git stash is useful when:

- You need to switch branches but don't want to commit your changes.
- You want to work on a different task without committing your current changes.
- You need to revert to a previous state without losing your changes.

Remember, git stash only saves changes to tracked files. Untracked files are not included in the stash. Use git stash -u or git stash --include-untracked to include untracked files.



Git reflog

git reflog is a command that displays a log of all changes made to the local repository's references (branches, tags, etc.). It shows a record of all commits, including those that have been deleted or overwritten.

Common uses:

1. View reflog:

- git reflog: Display the reflog.

2. View reflog with details:

- git reflog -p: Display the reflog with commit details.

3. View reflog for a specific branch:

- git reflog <branch-name>: Display the reflog for the specified branch.

git reflog is useful for:

- Recovering deleted commits or branches
- Understanding the commit history
- Finding the commit hash of a lost commit
- Checking the order of commits

Reflog entries are stored for a certain period (default 90 days) and can be configured using git config.

Some key terms:

- HEAD@{<index>}: Refers to the commit at a specific position in the reflog.
- HEAD@{<timestamp>}: Refers to the commit at a specific timestamp.

By using git reflog, you can easily recover from mistakes, like deleted commits or branches, and understand the evolution of your repository.

`git add .` :- add all files to staging area

`git commit -m "MESSAGE"` :- commits the staged changes to local repository.

`git log` :- show commit history.

use "`git rm --cached <file> ...`" to unstage particular file

Git is a version control system that is used to track changes to your files.

GIT is a software and can be installed on your computer.

Github is a web-based hosting service for Git repositories. Github is an online platform that allows you to store and share your code with others.